

Mathematical formalization with Lean 4

Lecture 1

María Inés de Frutos-Fernández

Universität Bonn

14 September 2026

AI and Mathematics Days at ICMAT

On today's lecture

- What mathematical formalization is.

On today's lecture

- What mathematical formalization is.
- What mathematical formalization is *not*.

On today's lecture

- What mathematical formalization is.
- What mathematical formalization is *not*.
- Why formalize mathematics (spoiler: not just for correctness).

On today's lecture

- What mathematical formalization is.
- What mathematical formalization is *not*.
- Why formalize mathematics (spoiler: not just for correctness).
- Lean, Mathlib and the Lean ecosystem.

On today's lecture

- What mathematical formalization is.
- What mathematical formalization is *not*.
- Why formalize mathematics (spoiler: not just for correctness).
- Lean, Mathlib and the Lean ecosystem.
- Formalization and AI.

On today's lecture

- What mathematical formalization is.
- What mathematical formalization is *not*.
- Why formalize mathematics (spoiler: not just for correctness).
- Lean, Mathlib and the Lean ecosystem.
- Formalization and AI.
- Validating a Lean proof.

Mathematical Formalization

- Digitize mathematical definitions, statements and proofs.
- **Proof assistant:**
 - Software that checks correctness of mathematical proofs.
 - It checks logical statements against a set of inference rules and some basic axioms.
 - [Lean](#), Rocq (Coq), Isabelle/HOL, Agda, Automath, Metamath, Mizar, ...

Example

The composition of two continuous functions is continuous.

```
import Mathlib.Topology.ContinuousFunction.Basic

open Function Set

lemma continuous_comp {X Y Z : Type} [TopologicalSpace X]
  [TopologicalSpace Y] [TopologicalSpace Z]
  {f : X → Y} {g : Y → Z} (hf : Continuous f)
  (hg : Continuous g) :
  Continuous (g ∘ f) := by
  rw [continuous_def] at hf hg ⊢
  intro S hS
  apply hf (g-1 S)
  apply hg S
  exact hS
```

▼ Tactic state

1 goal

X Y Z : Type

inst τ^2 : TopologicalSpace X

inst τ^1 : TopologicalSpace Y

inst τ : TopologicalSpace Z

f : X → Y

g : Y → Z

hf : $\forall (s : \text{Set } Y), \text{IsOpen } s \rightarrow \text{IsOpen } (f^{-1} s)$

hg : $\forall (s : \text{Set } Z), \text{IsOpen } s \rightarrow \text{IsOpen } (g^{-1} s)$

S : Set Z

hS : IsOpen S

⊢ IsOpen (g⁻¹ S)



Formalization News (I)

ANTHROPIC

[Research](#) ▾ [Policy](#) [Commitments](#) ▾ [Learn](#) ▾ [News](#)

Science

Formalizing Fermat's Last Theorem

Sep 4, 2026



Formalization News (II)

OpenAI



Log in ▾

Try ChatGPT ↗

September 8, 2026 Research Publication

On the Navier–Stokes Millennium Prize Problem

[Read the paper](#)[Link to Lean formalized proof >](#)

Formalization News (III)

AI authored projects > con-leche, a lean checker with a proof of consistency     SEP 10



Joachim Breitner EDITED

   11:08 AM

From the agentic creators of “The summer of soundness bugs”, and in cooperation with the Lean FRO, we present “The autumn of sound checkers”, episode 1:

I'm happy to announce [con-leche](#).

It is an external lean checker implementation with a consistency proof: If it accepts a set of lean declarations, then they have a set theoretical model (in ZFC with enough inaccessible). In particular, no theorem of `False` will be accepted.

Common misconceptions

A proof assistant...

... is not like LaTeX.

- The assistant checks the correctness of the proof.

Common misconceptions

A proof assistant...

... is not like LaTeX.

- The assistant checks the correctness of the proof.

... is not like Sage or Magma.

- Proofs, not computations.

Common misconceptions

A proof assistant...

... is not like LaTeX.

- The assistant checks the correctness of the proof.

... is not like Sage or Magma.

- Proofs, not computations.

... AI that proves theorems.

- Although such AI can be built on top of proof assistants.

Common misconceptions

A proof assistant...

... is not like LaTeX.

- The assistant checks the correctness of the proof.

... is not like Sage or Magma.

- Proofs, not computations.

... AI that proves theorems.

- Although such AI can be built on top of proof assistants.

... does not require programming experience for writing mathematical proofs.

- But of course it can be helpful.

Formalization is not AI

- AI can be used to write Lean code.
- Lean can be used to verify AI produced proofs.
- Using AI is neither sufficient nor necessary to formalize mathematics.
- ‘Vibe-coding’ without understanding is not recommended.

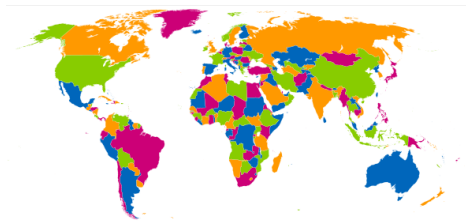
Formalization challenges

Formalization is **more than translating** informal mathematics to a computer language.

- Definitions need to be both correct and **efficient to use**.
- Proof arguments often need to be adapted.
- There are **omitted proofs**, and ‘folklore’ results without published proofs.
- Most references contain ‘invisible mathematics’ (e.g., treating isomorphic objects as equal, not checking commutativity of diagrams).
- Sometimes published references contain nontrivial **mistakes**.

Why formalize mathematics?

Checking Proof Correctness (I)



G. Gonthier (2005), *A computer-checked proof of the four colour theorem.*



T. Hales et. al. (2017), *A formal proof of the Kepler conjecture.*

Checking Proof Correctness (II)



Terence Tao

@tao@mathstodon.xyz

The #Lean4 formalization of the proof of the PFR conjecture has uncovered an extremely minor error in the paper itself; see github.com/teorth/pfr/pull/53. The issue is that in Lemma 5.2, the hypothesis that G is a vector space over F_2 (which is actually true throughout the paper) was omitted in the statement of this lemma, but is a needed hypothesis (in the proof one wants to rewrite $X - Y - Z$ as $X + Y + Z$, and this is only possible in characteristic 2). Thanks to Floris van Doorn for uncovering this issue. The fix is immediate (just put this omitted hypothesis back in), but it does show (for the second time, in my case) that formalization can uncover (small) errors in the original proof being formalized.

Checking Proof Correctness (III)

“Antoine Chambert-Loir and María Inés de Frutos Fernández have been teaching the theory of divided powers to Lean, and over the summer Lean did that irritating thing which it sometimes does: it complained about the human presentation of an argument in the standard literature, and on closer inspection it turned out that the human argument left something to be desired. **In particular a key lemma in Roby’s work seems to be incorrect.** [...] According to the way I currently view mathematics (as a formalist), the entire theory of crystalline cohomology vanished from the literature at the moment Antoine discovered the issue, with massive collateral damage [...]).”

– Kevin Buzzard ([Xena blog](#))

We **repaired and formalized** the construction of the universal divided power algebra.
A. Chambert-Loir, M. I. de Frutos-Fernández, [Formalizing polynomial laws and the universal divided power algebra](#), CPP 2026.

Two meanings of formalization

Formalization might mean two things:

Two meanings of formalization

Formalization might mean two things:

- **Formal verification**: writing a full, compiling proof of a statement in a proof assistant (e.g., Lean). This only gives a certificate of proof correctness.

Two meanings of formalization

Formalization might mean two things:

- **Formal verification**: writing a full, compiling proof of a statement in a proof assistant (e.g., Lean). This only gives a certificate of proof correctness.
- **Canonization**: obtain a formal verification for which both the main results and the prerequisites are **reusable, general, maintainable, efficient** and **extensible**.
 - Building libraries of formalized mathematics fits into canonization.
 - Can be extremely difficult and time-consuming.
 - Requires expert knowledge (both in formalization and in the content being formalized).
 - Less incentives if there is an existing formalization (even a poor quality one).

Formal verification vs canonization

AI autoformalizations provide formal verifications of proofs, but not canonizations (at least for now). For instance, consider Anthropic's autoformalization of Fermat's Last Theorem:

- 13 million lines of Lean code ($\sim 500,000$ pages) and 29,500 theorems.
- Huge de Bruijn factor (likely in the hundreds).
- For comparison, Lean's mathematical library consists of 2,5 million lines.
- FLT Lean proof is $\sim 5x$ as big as Mathlib, but takes $\sim 20x$ as long to compile.
- Code far from Mathlib style/quality.

Better Understanding

- “One key problem I had when I was trying to find this proof was that I was essentially unable to keep all the objects in my ‘RAM’, and I think the same problem occurs when trying to read the proof”.
- “When I wrote the blog post half a year ago, I did not understand why the argument worked (...). But (...) the formalization (...) made me realize that actually the key thing happening is a reduction from a non-convex problem over the reals to a convex problem over the integers”.

– Peter Scholze

Collaborative formalization projects (I)

- Individual contributors work on their own part of the proof.
- Statements are formalized by experts to ensure correct translation.
- No need to trust proof authors, since Lean checks the proofs.
- Lean guarantees consistency between project parts.

Collaborative formalization projects (II)

Blueprints to organize large formalizations:

Carleson operators on doubling metric measure spaces

Theorem 1.0.1. (classical Carleson) ✓ # 🤖 ✖️ LEAN

Let f be a 2π -periodic complex-valued continuous function on $\mathbb{R}$. Then for almost all $x \in \mathbb{R}$ we have

$$\lim_{N \rightarrow \infty} S_N f(x) = f(x), \quad (1.0.3)$$

where $S_N f$ is the N -th partial Fourier sum of f defined in (1.0.2).

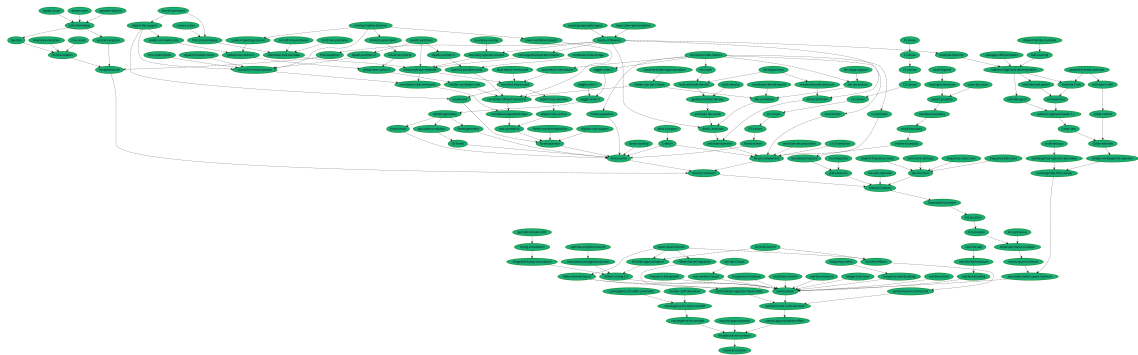
The purpose of this paper is, on one hand, to prepare computer verification of Theorem 1.0.1 by formalizing the proof as a blueprint for coding in Lean. We pass through a bound on the so-called Carleson operator to doubling metric measure spaces and proving these bounds constitutes the second purpose of this paper. This generalization incorporates several results from the

Lean declarations ✖️

`classical_carleson`

Built using P. Massot's [blueprint infrastructure](#).

Collaborative formalization projects (III)



Dependency graph from the [Carleson project](#).

Semantic Search Engines

Query
Name or description of the theorem or definition you are looking for

adele ring

Number of results
50

Clear
Query Augmentation
Search

Tip: Query Augmentation augments your query to increase the chance to find relevant results.

NumberField.AdeleRing

definition

$(R\ K : \text{Type}^*)$ [CommRing R] [IsDedekindDomain R] [Field K] [Algebra R K] [IsFractionRing R K]

► Adèle Ring of a Number Field

Doc
Doc-Next-Gen (beta)
Similar

↑ ↓ ×

LeanSearch

Loogle

Refactoring and generalization

- Safe refactoring: when a definition or theorem is reformulated, Lean will inform you of all the places that have to be adapted.
- Automated checks for unused variables.
- Principles of abstraction and generalization that sometimes lead to improved results.

Teaching and learning: controlled natural language

Exercise "Continuity implies sequential continuity"

Given: $(f : \mathbb{R} \rightarrow \mathbb{R}) (u : \mathbb{N} \rightarrow \mathbb{R}) (x_0 : \mathbb{R})$

Assume: $(hu : u \text{ converges to } x_0) (hf : f \text{ is continuous at } x_0)$

Conclusion: $(f \circ u) \text{ converges to } f x_0$

Proof:

Let's prove that $\forall \varepsilon > 0, \exists N, \forall n \geq N, |f (u n) - f x_0| \leq \varepsilon$

Fix $\varepsilon > 0$

By hf applied to ε using that $\varepsilon > 0$ we get δ such that

$\delta_pos : \delta > 0$ and $Hf : \forall x, |x - x_0| \leq \delta \Rightarrow |f x - f x_0| \leq \varepsilon$

By hu applied to δ using that $\delta > 0$ we get N such that

$Hu : \forall n \geq N, |u n - x_0| \leq \delta$

Let's prove that N works : $\forall n \geq N, |f (u n) - f x_0| \leq \varepsilon$

Fix $n \geq N$

By Hf applied to $u n$ it suffices to prove $|u n - x_0| \leq \delta$

We conclude by Hu applied to n using that $n \geq N$

QED

[Verbose Lean](#) by Patrick Massot.

Also: [Waterproof](#) by J. Wemmenhove et al.

User interface: ProofWidgets

The screenshot displays the ProofWidgets user interface, which is divided into two main panels.

Left Panel (Code Editor): Shows a Lean proof script in a file named `CommDiag.lean`. The code defines an example with types X, Y, Z and functions $f : X \rightarrow Y$, $g : Y \rightarrow Z$, and $h : X \rightarrow Z$. It states the goal $f \circ g = h$ and provides a proof using `withSelectionDisplay`, `intro h1, h2`, and `rw [← h1, h2]`.

Right Panel (Lean Infview): Displays the current state of the proof. It shows the tactic state with the goal $h = f \circ g \rightarrow i \circ j = h \rightarrow f \circ g = i \circ j$. Below this, it shows the selected expressions, which are graphical representations of the functions f , g , and h as nodes in a directed graph. The graph shows the composition of functions f and g to form $f \circ g$, and the function h mapping X to Z . The goal is to show that $f \circ g = h$.

ProofWidgets by W. Nawrocki, E. W. Ayers, and G. Ebner.

Communication: a new kind of mathematical document

Theorem (continuous_of_dense). Let X be a topological space and let Y be a regular topological space. Let A be a dense subset of X . Let $f : X \rightarrow Y$ be a function. Assume that for all elements x of X , f is continuous at x within A . Then f is continuous.

Proof. $\bullet \oplus$ Let x be an element of X . $\circ$ One can see it suffices to prove that for all closed neighborhoods V' of $f(x)$, there exists a neighborhood U of x such that $f[U] \subseteq V'$. $\oplus \circ$ Let V' be a closed neighborhood of $f(x)$. $\circ \oplus$ Using our assumption that for all elements x' of X , f is continuous at x' within A and our assumption that V' is a neighborhood of $f(x)$, we obtain an open neighborhood V of x such that $f[V \cap A] \subseteq V'$. $\circ$ We will show that V is suitable by proving that V is a neighborhood of x and $f[V] \subseteq V'$. By assumption, V is a neighborhood of x . $\circ$ Let z be an element of V . $\circ \oplus$ One can see that $f(z_1)$ tends to $f(z)$ as z_1 tends to z within $V \cap A$. $\circ$

Consider the following:

$f(z) \in \overline{f[V \cap A]}$	$\oplus$ Using our <u>assumption</u> that $f(z_1)$ tends to $f(z)$ as z_1 tends to z within $V \cap A$ proves this step.
$\subseteq \overline{V'}$	$\oplus$ Using our assumption that $f[V \cap A] \subseteq V'$ proves this step.
$= V'$	$\oplus$ Using our assumption that V' is closed proves this step.

This completes the proof. $\square$

Current proof state:

X is a topological space

Y is a regular topological space

A is a dense subset of X

(hA)

$f : X \rightarrow Y$

For all elements x of X , f is continuous at x within A

(hf)

Goal: f is continuous

Interactive document by P. Massot, K. Miller.

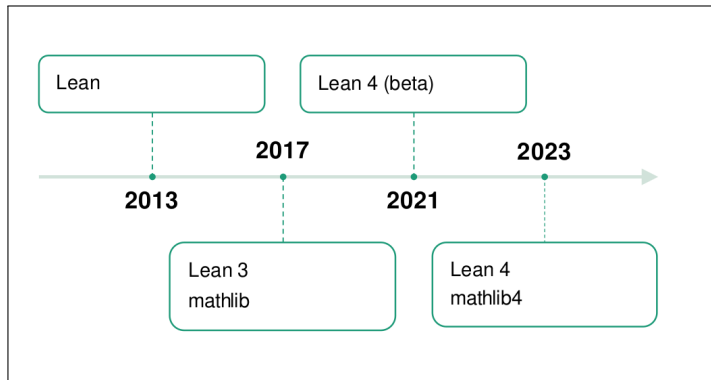
Summary: Why formalize mathematics?

- Check proof correctness.
- Canonization.
- Better understanding.
- New forms of collaboration.
- Semantic search engines for mathematical statements.
- Teaching, learning and communication of mathematics.
- All of these require extensive databases of statements and proofs.

Lean and Mathlib

Lean

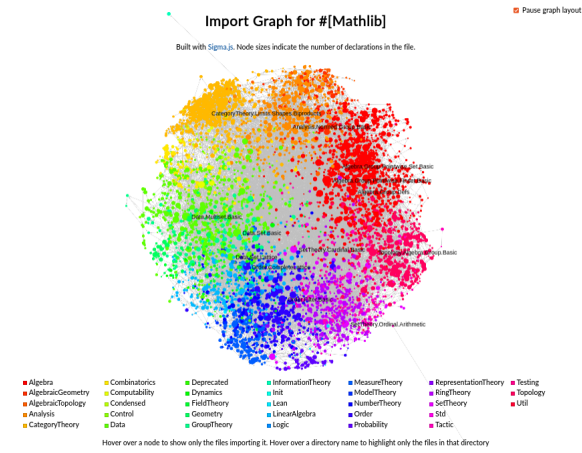
Lean is an interactive theorem prover and a functional programming language. It was launched by Leonardo de Moura at Microsoft Research.



Mathlib

- The mathematics library of Lean is called [Mathlib](#).
- Monolithic.
- Open source, community-driven effort (772 contributors).
- Generality in definitions and statements.
- Thorough peer-review process (62 reviewers, including 28 maintainers).
- Many contributors have a background in pure mathematics.
- $\approx 136\text{K}$ definitions. $\approx 288\text{K}$ theorems. $\approx 2.5\text{M}$ lines.

Mathlib's Import Graph



Interactive version.

Learning resources

- The next two lectures will be an introduction to Lean and Mathlib.
- [Online resources](#) for Learning Lean 4.
 - [Mathematics in Lean](#): standard mathematics-oriented reference.
 - [Theorem Proving in Lean 4](#): bigger focus on Lean's type-theory foundations.
 - [The Lean Language Reference](#): reference manual.
- Lean and Mathlib [documentation overview](#).
- Lean community [Zulip chat](#).

(Some) Large Scale Projects in Lean

- [Perfectoid spaces](#) (Buzzard, Commelin, Massot; 2020).
- [Liquid Tensor Experiment](#) (led by Commelin and Topaz; 2022).
- [Sphere Eversion](#) (van Doorn, Massot, Nash; 2022).
- [Polynomial Freiman – Ruzsa conjecture](#) (led by Tao; 2023).
- [Prime Number Theorem](#) (led by Kontorovich and Tao; 2025).
- [Carleson's Theorem](#) (led by van Doorn; 2025).
- [Sphere packing](#) (Birkbeck, Hariharan, Ma, Mehta, Lee, Viazovska; 2026)
- [Fermat's Last Theorem](#) (led by Buzzard; Ongoing).

My Formalization Projects

- [Ring of Adèles of a Global Field](#) (2022).
- [Norm extensions and \$\mathbb{C}_p\$](#) (2023).
- [Complete Discrete Valuation Rings and Local Fields](#), with Filippo Nuccio (2024).
- [Divided powers](#), with Antoine Chambert-Loir (2025).
- [Polynomial laws and the universal divided power algebra](#), with Antoine Chambert-Loir (2026).
- The divided power envelope and B_{crys} , with Antoine Chambert-Loir (Ongoing).
- Drinfeld modules, with Xavier G  n  reux (Ongoing).
- [Carleson's Theorem](#), 14 authors (2025).

The Lean formal mathematics ecosystem

Mathlib is the foundation for:

- [Formal conjectures](#): repository of formalized mathematical conjectures.
- [LeanEval](#): Lean benchmark for formal mathematics. Submissions can remain private.
- [Tau Ceti](#): AI-generated library of formal mathematics (building on and following Mathlib).
- [Palomar](#): aims to index significant Lean formalizations: clear main statements, robustly audited. Could become 'arXiv for Lean'.

Formalization and AI

Artificial Intelligence: IMO

2024 International Mathematics Olympiad

- Silver medal for Google DeepMind (AlphaGeometry + AlphaProof)

2025 International Mathematics Olympiad

- Gold medal for Harmonic (Aristotle), using Lean 4.
- Silver medal for ByteDance (SeedProver), using Lean 4.
- Gold medals for Google DeepMind and OpenAI, using natural language approach.

Artificial Intelligence: autoformalization agents

Several companies provide access to autoformalization agents.

- [AlphaProof](#), by Google DeepMind.
- [Aristotle](#), by Harmonic.
- [OpenGauss](#), by Math Inc.

Human-AI joint projects:

- Proof that the [abc conjecture is true almost always](#), using Trinity by Morph Labs (2025).
- Proof of the [strong Prime Number Theorem](#), using Gauss (2025).
- The [sphere packing](#) project contains Lean code co-authored by Aristotle and Gauss (2026).

Some notable AI formalizations

- [Feit–Thompson odd-order theorem](#) (using various models)
- Counterexample to the [Erdos unit distance conjecture](#) (OpenAI).
- Counterexample to the [Jacobian Conjecture](#) (using Claude).
- [Riemann Hypothesis](#) lower-bound improvement (Anthropic).
- [Fermat's Last Theorem](#) (Anthropic).
- [Navier–Stokes](#) solution (OpenAI).

Validating a Lean proof

Validating a Lean proof (I)

Distinguish between honest and possibly malicious formalizations.

Warning: do not build untrusted Lean code locally.

Is the Lean code formalizing what it claims to?

- Carefully check definitions, notations and statements.
- Use Mathlib definitions where possible.

Validating a Lean proof (II)

Escalating sequence of checks:

- **Blue double check marks**: proof accepted by Lean kernel, using definitions, theorems and axioms declared in current file and its imports.
- **#print axioms**: should return a subset of propext, Classical.choice, and Quot.sound.
- **Comparator tool**: verifies a proof against a 'challenge' file with a main statement.
- **External checkers**: nanoda, lean4lean, con-leche, others in Lean Kernel Arena.

Questions

Thanks for listening! Questions?

<https://mariainesdff.github.io/>